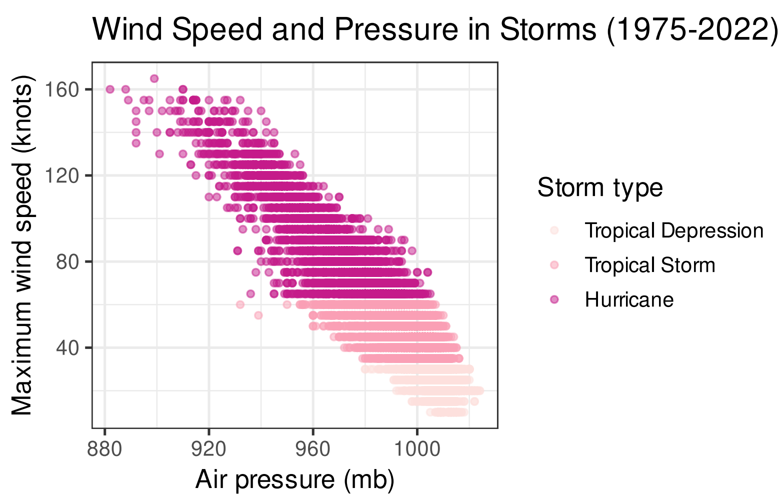
Explanatory Data Analysis

All of the plots in this problem set are meant to focus on a particular story to tell your a particular audience. As you craft them, think carefully about the labels, themes, aesthetic settings, and the manner in which it facilitates comparisons.

1. Provide the code that will create this plot from the `storms` data set inside the `dplyr` package. Here, each dot is a single storm. A few notes:

- You will need to create a new column indicating storm type. For that you'll need to learn how that determination is made and then add it to the data frame. The `case_when()` function is a helpful shortcut similar to `ifelse()`.
- The color palette belows to a set of palettes called ColorBrewer. You can read more about them here: https://ggplot2.tidyverse.org/reference/scale_brewer.html.



2. Are there any changes that you'd make to the plot above to make it easier to read or clarify what it is showing?

3. Remake your plot using a very specific visual theme: the botanical paintings of the US Department of Agriculture. They have been wrapped up into a ggplot2 theme inside the `ggpomological` package: https://github.com/gadenbuie/ggpomological. Read about how to use the package and write below the additional layers that you added to your plot above (for a challenge, see if you can get the handwriting font to show up).

4. The `present` data frame stores data from the US Census about the number of boys and girls born each year between 1940 and 2013. It is stored in the `stat20data` package, which can be installed from GitHub using the `devtools` package.

   ```r
   # install.packages("devtools")
   # devtools::install_github("https://github.com/stat20/stat20data")
   library(stat20data)
   slice(present, 1:5)
   ```

   Make a plot to visualize the trend over time in the proportion of births that are girls. Put the annual variability into context by ensuring the viewer can see the full range of possible values that the proportion could take. Allow the comparison to a baseline of .5 by drawing a horizontal line at that value (see `geom_hline()`).

5. (Challenge) Some of the most effective plots on the web don't come from R or Python, but from dedicated visualization libraries written in JavaScript. That is the case for the EV registration plot made by the Department of Energy (https://afdc.energy.gov/data/10962), which uses a library called Highcharts.

   While you could craft the plot directly using JavaScript, a member of the R community has written a package that allows you to make them directly from R: you write R code, which the package turns into Javascript, which Highcharts then renders as a plot. Read how to use the package on its documention site: https://github.com/jbkunst/highcharter/.

   Your task is to recreate the EV Highcharts plot exactly using the data found in an excel spreadsheet on the DoE website. You can export the excel spreadsheet as a csv file, as we did in class, or you can experiment with the `readxl` package to read it into R directly.

Joins

```
# A tibble: 4 × 3
  student_id name    major
       <dbl> <chr>   <chr>
1          1 Ada     Stat
2          2 Bruno   DS
3          3 Chandra Stat
4          4 Diego   Econ
```

```
# A tibble: 4 × 2
  student_id quiz_score
       <dbl>      <dbl>
1          2         88
2          3         94
3          4         77
4          5         91
```

These data sets illustrate a roster of students registered for a particular lab section (`roster`) and the record of grades on a quiz (`grades`). For each of the following, write appropriate the `dplyr` join command and sketch the resulting data frame that contains those records.

6. All registered students who took the quiz.

7. The quiz scores for all registered students.

8. The names and majors or all students who took the quiz.

9. All records for students either registered for the section or who took the quiz or both.

10. The name and major of the registered students who didn't take the quiz.

11. Calculate the maximum wind speed of each storm, then attach that column of data to the original `storms` data frame.

12. Create a small data frame that you can then use to filter the `storms` data so that it only retains records of storms that are "Hurricanes", "Tropical Storms", and "Tropical Depressions". (note: don't use `filter`!)

13. Create a small data frame that you can use to add on a new column to `storms` called `type` that has levels of "Hurricanes", "Tropical Storms", and "Tropical Depressions" for storms with that `status` and "Other" for all other storms. (note: don't use `mutate()`!)

14. Remake the state-by-state EV registration chart but instead of plotting the distribution of registrations, display, for every state, either: a) the proportion of vehicle registrations that are EV b) the ratio of EV registrations to residents. To do this you will need to track down additional data online and join it with the EV data registration data.

15. In the space below, sketch out four Venn diagrams, one for each of the mutating joins. Each diagram should have two circles representing tables X and Y, showing some overlap. Shade in the parts of the diagram where those rows exist in the joined data frame.